

Tensorflow II

Convolutional Neural Networks (Continued)

Fatemeh Dehrouyeh



1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- Parameters in a CNN
- Building a Complete CNN
- Compiling a CNN
- Training a CNN
 - CIFAR-10 dataset
- Monitoring Training

Current Section

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- Parameters in a CNN
- Building a Complete CNN
- Compiling a CNN
- Training a CNN
- Monitoring Training

Current Subsection

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- Parameters in a CNN
- Building a Complete CNN
- Compiling a CNN
- Training a CNN
- Monitoring Training

What Is a Receptive Field?

The **receptive field** is the region of the input image that affects one feature value in a later layer.

In early layers, a neuron sees only a small local region.

In deeper layers, a neuron can indirectly see a larger part of the image.

Key Idea

Stacking convolution and pooling layers allows CNNs to combine small local patterns into larger visual structures.

Watch This Short Video

Code

```

import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(224, 224, 3)),
    tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.summary()

```

... Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 16)	448
max_pooling2d (MaxPooling2D)	(None, 112, 112, 16)	0
conv2d_1 (Conv2D)	(None, 112, 112, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 32)	0
flatten (Flatten)	(None, 100352)	0
dense (Dense)	(None, 10)	1,003,530

Total params: 1,008,618 (3.85 MB)

Trainable params: 1,008,618 (3.85 MB)

Non-trainable params: 0 (0.00 B)

Deeper Layers Learn More Abstract Features

A CNN learns features hierarchically.

- Early layers: edges, corners, simple colors
- Middle layers: textures, repeated patterns, object parts
- Deeper layers: higher-level object structures

Pixels $\rightarrow$ Edges $\rightarrow$ Textures $\rightarrow$ Parts $\rightarrow$ Objects

Important

This hierarchy is learned from data during training.

Current Subsection

1 Convolutional Neural Networks

- Receptive Field
- **Flattening and Dense Layers**
- Output Layers
- Parameters in a CNN
- Building a Complete CNN
- Compiling a CNN
- Training a CNN
- Monitoring Training

From Feature Maps to Prediction

After convolution and pooling layers, the network has feature maps.

To make a prediction, these features must be converted into output scores.

A common pipeline is:

Feature Maps $\rightarrow$ Flatten $\rightarrow$ Dense Layer $\rightarrow$ Prediction

Example

For classification, the final layer usually outputs one score per class.

What Does Flatten Do?

Flatten converts a multi-dimensional feature map into one long vector.

For example:

$$7 \times 7 \times 64 \rightarrow 3136$$

This vector can then be passed to a dense layer.

Important

Flattening can create many parameters if the feature map is still large.

Global Average Pooling

Instead of flattening, many modern CNNs use **global average pooling**.

It averages each feature map into one number:

$$H \times W \times C \rightarrow C$$

For example:

$$7 \times 7 \times 64 \rightarrow 64$$

Why Use It?

Global average pooling usually reduces the number of parameters compared with flattening.

Code

```

import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(224, 224, 3)),

    tf.keras.layers.Conv2D(16, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(32, 3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.GlobalAveragePooling2D(),

    tf.keras.layers.Dense(10, activation="softmax")
])

model.summary()

```

... Model: "sequential_2"

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 224, 224, 16)	448
max_pooling2d_4 (MaxPooling2D)	(None, 112, 112, 16)	0
conv2d_5 (Conv2D)	(None, 112, 112, 32)	4,640
max_pooling2d_5 (MaxPooling2D)	(None, 56, 56, 32)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 32)	0
dense_2 (Dense)	(None, 10)	330

Total params: 5,418 (21.16 KB)
 Trainable params: 5,418 (21.16 KB)
 Non-trainable params: 0 (0.00 B)

Current Subsection

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- **Output Layers**
- Parameters in a CNN
- Building a Complete CNN
- Compiling a CNN
- Training a CNN
- Monitoring Training

Output Layer for Classification

For image classification, the output layer depends on the number of classes.

For K classes, the final layer usually has:

$$K$$

neurons.

For multi-class classification, we often use:

```
Dense(K, activation="softmax")
```

Interpretation

Softmax converts output scores into class probabilities.

Softmax Output

Softmax produces a probability distribution over classes.

For example, if we have 3 classes:

$$[2.0, 1.0, 0.1] \rightarrow [0.66, 0.24, 0.10]$$

The largest probability becomes the predicted class.

Key Idea

The network outputs scores first; softmax turns those scores into probabilities.

Current Subsection

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- **Parameters in a CNN**
- Building a Complete CNN
- Compiling a CNN
- Training a CNN
- Monitoring Training

What Are Trainable Parameters?

A CNN learns numerical parameters during training.

The main trainable parameters are:

- convolution filter weights,
- convolution biases,
- dense-layer weights,
- dense-layer biases.

Key Idea

Training means adjusting these parameters so that the model makes better predictions.

How Many Parameters Does a Conv2D Layer Have?

For a convolution layer:

$$\text{Parameters} = (K_h K_w C_{\text{in}} + 1) \times C_{\text{out}}$$

where:

- K_h, K_w : kernel height and width,
- C_{in} : input channels,
- C_{out} : number of filters,
- $+1$: one bias per filter.

Example

For 3×3 kernels, 3 input channels, and 16 filters:

$$(3 \cdot 3 \cdot 3 + 1) \cdot 16 = 448$$

parameters.

Current Subsection

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- Parameters in a CNN
- **Building a Complete CNN**
- Compiling a CNN
- Training a CNN
- Monitoring Training

A Small CNN Architecture

A simple image classifier can be built as:

Input $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ GAP $\rightarrow$ Dense

Design Pattern

As spatial resolution decreases, the number of feature maps often increases.

Code

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(128, 128, 3)),

    tf.keras.layers.Conv2D(
        32, 3,
        padding="same",
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(
        64, 3,
        padding="same",
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D(),

    tf.keras.layers.Conv2D(
        128, 3,
        padding="same",
        activation="relu"
    ),

    tf.keras.layers.GlobalAveragePooling2D(),

    tf.keras.layers.Dense(
        10,
        activation="softmax"
    )
])

model.summary()
```

Code—Result

... Model: "sequential_3"

Layer (type)	Output Shape	Param #
conv2d_7 (Conv2D)	(None, 128, 128, 32)	896
max_pooling2d_4 (MaxPooling2D)	(None, 64, 64, 32)	0
conv2d_8 (Conv2D)	(None, 64, 64, 64)	18,496
max_pooling2d_5 (MaxPooling2D)	(None, 32, 32, 64)	0
conv2d_9 (Conv2D)	(None, 32, 32, 128)	73,856
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 128)	0
dense_3 (Dense)	(None, 10)	1,290

Total params: 94,538 (369.29 KB)

Trainable params: 94,538 (369.29 KB)

Non-trainable params: 0 (0.00 B)

Understanding the Shape Changes

For an input image of

$$128 \times 128 \times 3$$

a CNN may transform the data as:

$$128 \times 128 \times 3$$



$$128 \times 128 \times 32$$



$$64 \times 64 \times 32$$



$$64 \times 64 \times 64$$



$$32 \times 32 \times 64$$

Pattern

Spatial dimensions shrink, while feature depth usually increases.

Current Subsection

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- Parameters in a CNN
- Building a Complete CNN
- **Compiling a CNN**
- Training a CNN
- Monitoring Training

Before Training: Compile the Model

Before training, TensorFlow needs three main choices:

- **optimizer**: how parameters are updated,
- **loss function**: how prediction error is measured,
- **metrics**: how performance is reported.

For multi-class image classification, a common setup is:

Optimizer = Adam

Loss = Sparse Categorical Crossentropy

Metric = Accuracy

What Is the Loss Function?

The loss function measures how wrong the model's predictions are.

During training:

Prediction $\rightarrow$ Loss $\rightarrow$ Parameter Update

A good model should make the loss decrease over time.

Key Idea

The optimizer uses the loss to determine how the model parameters should change.

What Does the Optimizer Do?

The optimizer updates model parameters to reduce the loss.

Conceptually:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \nabla_{\theta} L$$

where:

- θ : model parameters,
- L : loss,
- η : learning rate.

Popular Optimizer

Adam is commonly used as a strong default optimizer.

Current Subsection

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- Parameters in a CNN
- Building a Complete CNN
- Compiling a CNN
- **Training a CNN**
 - CIFAR-10 dataset
- Monitoring Training

Training the Model

Training repeatedly shows batches of images to the model.

For each batch:

Forward Pass $\rightarrow$ Loss $\rightarrow$ Backpropagation $\rightarrow$ Update

This process is repeated over many batches and epochs.

Current Subsubsection

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- Parameters in a CNN
- Building a Complete CNN
- Compiling a CNN
- **Training a CNN**
 - CIFAR-10 dataset
- Monitoring Training

What is the CIFAR Dataset?

- CIFAR stands for **Canadian Institute for Advanced Research**.
- It is a popular image dataset used in machine learning and computer vision.
- It is mainly used for training and testing image classification models.
- The most common versions are:
 - CIFAR-10
 - CIFAR-100

CIFAR-10 Overview

- CIFAR-10 contains **60,000 colour images**.
- Each image has a size of **32×32 pixels**.
- The dataset has **10 classes**.
- Each class contains **6,000 images**.
- It is divided into:
 - 50,000 training images
 - 10,000 test images

CIFAR-10 Classes

- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

CIFAR-100 Overview

- CIFAR-100 is similar to CIFAR-10 but more challenging.
- It also contains **60,000 colour images**.
- Each image is **32×32 pixels**.
- The dataset has **100 classes**.
- Each class contains **600 images**.
- The 100 classes are grouped into **20 superclasses**.

Why is CIFAR Useful?

- It is small enough to train quickly compared with large datasets.
- It is more complex than MNIST because it uses real colour images.
- It helps evaluate image classification algorithms.
- It is commonly used to test CNN models.
- It provides a balanced dataset, meaning each class has the same number of images.

CIFAR vs MNIST

Feature	MNIST	CIFAR-10
Image Type	Grayscale digits	Colour objects
Image Size	28×28	32×32
Classes	10	10
Difficulty	Easier	Harder
Main Task	Digit recognition	Object recognition

How CIFAR is Used in CNNs

- ➊ Load CIFAR image data.
- ➋ Normalize pixel values between 0 and 1.
- ➌ Pass images through convolutional layers.
- ➍ Extract visual features such as edges, shapes, and textures.
- ➎ Use dense layers to classify the image.
- ➏ Compare the predicted class with the true label.

Load the CIFAR-10 dataset

```
▶ import tensorflow as tf
  from tensorflow import keras
  from tensorflow.keras import layers
  import matplotlib.pyplot as plt

  (x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

  print("Training images:", x_train.shape)
  print("Training labels:", y_train.shape)
  print("Test images:", x_test.shape)
  print("Test labels:", y_test.shape)
```

... Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 1499s 9us/step
Training images: (50000, 32, 32, 3)
Training labels: (50000, 1)
Test images: (10000, 32, 32, 3)
Test labels: (10000, 1)

CIFAR-10 Samples

```
▶ class_names = [  
    "airplane",  
    "automobile",  
    "bird",  
    "cat",  
    "deer",  
    "dog",  
    "frog",  
    "horse",  
    "ship",  
    "truck"  
]  
  
plt.figure(figsize=(8, 8))  
  
for i in range(9):  
    plt.subplot(3, 3, i + 1)  
    plt.imshow(x_train[i])  
    plt.title(class_names[y_train[i][0]])  
    plt.axis("off")  
  
plt.show()
```

CIFAR-10 Samples—Results



Processing Data

```
# Normalize the images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Create training and validation datasets
train_ds = tf.data.Dataset.from_tensor_slices((x_train, y_train))
train_ds = train_ds.shuffle(50000).batch(32)

val_ds = tf.data.Dataset.from_tensor_slices((x_test, y_test))
val_ds = val_ds.batch(32)
```

from_tensor_slices() converts arrays into a dataset of individual (input, label) examples.

Defining Model

```

▶ input_shape = train_ds.element_spec[0].shape[1:]

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=input_shape),
    tf.keras.layers.Conv2D(32, kernel_size=3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Conv2D(64, kernel_size=3, padding="same", activation="relu"),
    tf.keras.layers.MaxPooling2D(pool_size=(2, 2)),
    tf.keras.layers.Conv2D(128, kernel_size=3, padding="same", activation="relu"),
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.summary()

```

... Model: "sequential_4"

Layer (type)	Output Shape	Param #
conv2d_10 (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d_6 (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_11 (Conv2D)	(None, 16, 16, 64)	18,496
max_pooling2d_7 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_12 (Conv2D)	(None, 8, 8, 128)	73,856
global_average_pooling2d_3 (GlobalAveragePooling2D)	(None, 128)	0
dense_4 (Dense)	(None, 10)	1,290

Total params: 94,538 (369.29 KB)

Trainable params: 94,538 (369.29 KB)

Non-trainable params: 0 (0.00 B)

Compiling Model

```
 model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

Now we can train the model

```
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=10  
)
```

```
*** Epoch 1/10  
1563/1563 ————— 107s 68ms/step - accuracy: 0.3598 - loss: 1.7218 - val_accuracy: 0.4703 - val_loss: 1.4657  
Epoch 2/10  
1563/1563 ————— 104s 67ms/step - accuracy: 0.4990 - loss: 1.3892 - val_accuracy: 0.5427 - val_loss: 1.2747  
Epoch 3/10  
1563/1563 ————— 104s 67ms/step - accuracy: 0.5606 - loss: 1.2277 - val_accuracy: 0.5585 - val_loss: 1.2307  
Epoch 4/10  
1563/1563 ————— 109s 70ms/step - accuracy: 0.5953 - loss: 1.1389 - val_accuracy: 0.5940 - val_loss: 1.1244  
Epoch 5/10  
1563/1563 ————— 109s 70ms/step - accuracy: 0.6233 - loss: 1.0600 - val_accuracy: 0.6055 - val_loss: 1.1377  
Epoch 6/10  
1563/1563 ————— 142s 69ms/step - accuracy: 0.6439 - loss: 1.0024 - val_accuracy: 0.6367 - val_loss: 1.0287  
Epoch 7/10  
1563/1563 ————— 105s 67ms/step - accuracy: 0.6646 - loss: 0.9514 - val_accuracy: 0.6719 - val_loss: 0.9433  
Epoch 8/10  
1563/1563 ————— 105s 67ms/step - accuracy: 0.6794 - loss: 0.9051 - val_accuracy: 0.6733 - val_loss: 0.9172  
Epoch 9/10  
1563/1563 ————— 105s 67ms/step - accuracy: 0.6956 - loss: 0.8637 - val_accuracy: 0.6895 - val_loss: 0.8818  
Epoch 10/10  
1563/1563 ————— 105s 67ms/step - accuracy: 0.7061 - loss: 0.8346 - val_accuracy: 0.6975 - val_loss: 0.8479
```

What Is an Epoch?

An **epoch** means that the model has processed the entire training dataset once.

If a dataset contains 10,000 images and the batch size is 100:

$$\frac{10,000}{100} = 100$$

batches are processed in one epoch.

Important

Training usually requires multiple epochs so the model can gradually improve.

What Is a Batch?

Instead of processing the entire dataset at once, training uses small groups called **batches**.

Example:

$$\text{Batch Size} = 32$$

means the model processes 32 images before updating its parameters.

Why Use Batches?

Batches make training computationally practical and provide efficient gradient estimates.

Training and Validation Data

Training data and validation data have different purposes.

- **Training set:** used to update model parameters.
- **Validation set:** used to evaluate generalization during training.

Important

Validation images must not be used to update the model weights.

Current Subsection

1 Convolutional Neural Networks

- Receptive Field
- Flattening and Dense Layers
- Output Layers
- Parameters in a CNN
- Building a Complete CNN
- Compiling a CNN
- Training a CNN
- **Monitoring Training**

Learning Curves

During training, we usually monitor:

Training Loss

Validation Loss

Training Accuracy

Validation Accuracy

Why Plot Them?

Learning curves help us detect underfitting, overfitting, and unstable training.

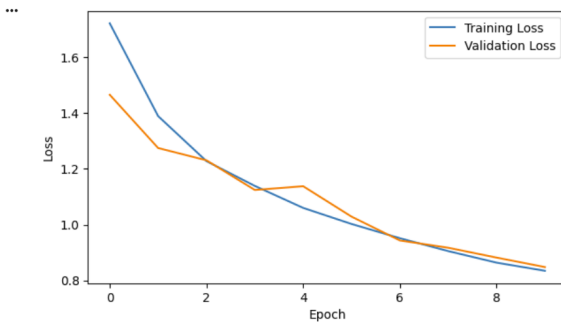
Code

```
import matplotlib.pyplot as plt

plt.figure(figsize=(7, 4))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.show()
```

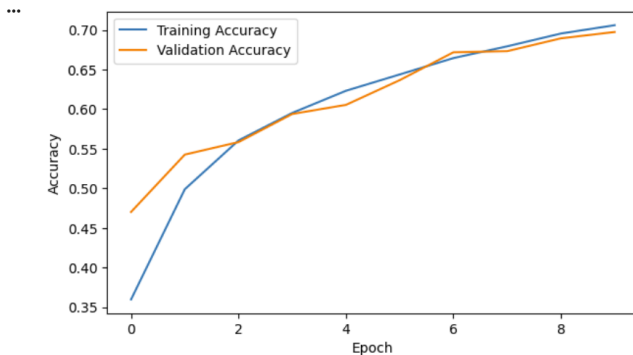


Code

```
plt.figure(figsize=(7, 4))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```



What Is Overfitting?

Overfitting occurs when the model performs very well on training data but does not generalize well to unseen data.

A common pattern is:

Training Loss ↓

while

Validation Loss ↑

Possible Solutions

Data augmentation, dropout, regularization, early stopping, and more training data can help reduce overfitting.

Testing the Model

To test the model, let's search for an arbitrary image belonging to one of the CIFAR-10 classes to see whether the model could classify it correctly.

I selected an image of a horse:



Result

```
▶ img = tf.keras.utils.load_img("horse.png", target_size=(32, 32))  
img = tf.keras.utils.img_to_array(img) / 255.0  
img = tf.expand_dims(img, axis=0)  
  
prediction = model.predict(img)  
predicted_class = class_names[tf.argmax(prediction[0])]  
  
print("Predicted class:", predicted_class)  
  
... 1/1 ————— 0s 172ms/step  
Predicted class: horse
```

Congratulations!

Now you know how to train a CNN!

